(i)

In [27]:
```
import numpy as np

T_half = 5730.0
decay_constant = np.log(2) / T_half

print(f"Decay Constant : {decay_constant:.6e} years⁻¹")
```

```
Decay Constant : 1.209681e-04 years⁻¹
```

(ii)

In [32]:
```
import numpy as np

T_half = 5730.0
decay_constant = np.log(2) / T_half
decay_probability = 1 - np.exp(-decay_constant*1)

print(f"Decay Probability (p): {decay_probability:.6e}")
```

```
Decay Probability (p): 1.209608e-04
```

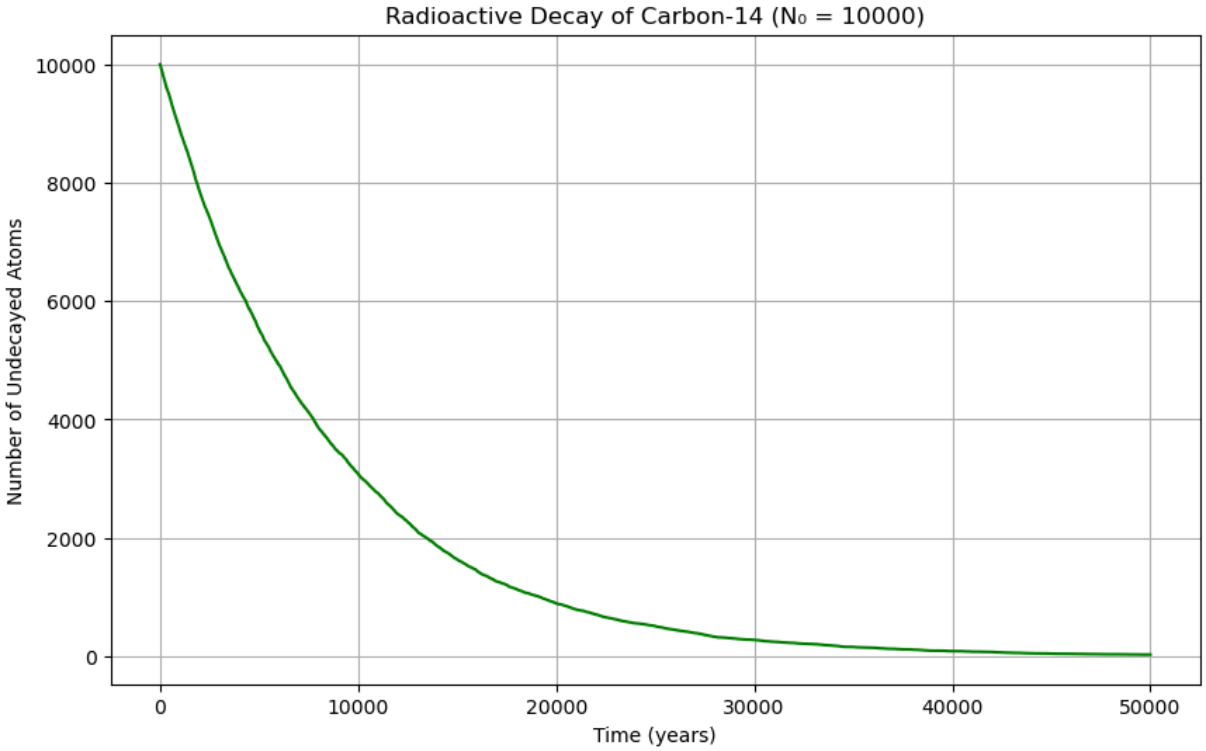
(iii)

In [33]:
```
import numpy as np
import random

T_half = 5730.0
decay_constant = np.log(2) / T_half
decay_probability = 1 - np.exp(-decay_constant*1)

def decay(N_init, probability, total_time):
    undecayed_atoms = [N_init]
    current_atoms = N_init

    for _ in range(1, total_time + 1):
        decayed_count = 0
        for _ in range(current_atoms):
            if random.random() < probability:
                decayed_count += 1

        current_atoms -= decayed_count
        undecayed_atoms.append(current_atoms)

    return undecayed_atoms
```

(iv)

In [31]:

```
import numpy as np
import matplotlib.pyplot as plt
import random

T_half = 5730.0
decay_constant = np.log(2) / T_half
decay_probability = 1 - np.exp(-decay_constant*1)

def decay(N_init, probability, total_time):
    undecayed_atoms = [N_init]
    current_atoms = N_init

    for _ in range(1, total_time + 1):
        decayed_count = 0
        for _ in range(current_atoms):
            if random.random() < probability:
                decayed_count += 1

        current_atoms -= decayed_count
        undecayed_atoms.append(current_atoms)

    return undecayed_atoms

N0 = 10000
total_years = 50000

decay = decay(N0, decay_probability, total_years)
t = list(range(total_years + 1))

plt.figure(figsize=(10, 6))
plt.plot(t, decay, color='green')
plt.title(f'Radioactive Decay of Carbon-14 (N₀ = {N0})')
plt.xlabel('Time (years)')
plt.ylabel('Number of Undecayed Atoms')
plt.grid()
plt.show()
```

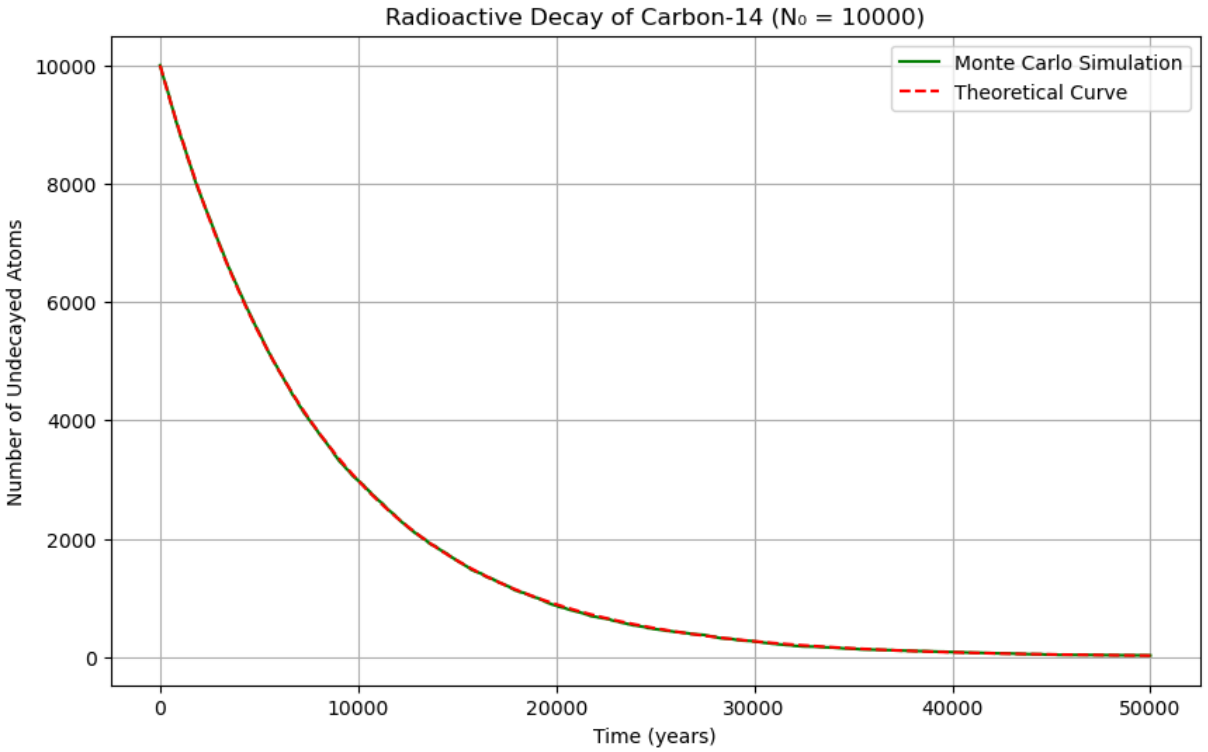


(v)

In [29]:

```python
import numpy as np
import matplotlib.pyplot as plt
import random

T_half = 5730.0
decay_constant = np.log(2) / T_half
decay_probability = 1 - np.exp(-decay_constant*1)

def decay(N_init, probability, total_time):
    undecayed_atoms = [N_init]
    current_atoms = N_init

    for _ in range(1, total_time + 1):
        decayed_count = 0
        for _ in range(current_atoms):
            if random.random() < probability:
                decayed_count += 1

        current_atoms -= decayed_count
        undecayed_atoms.append(current_atoms)

    return undecayed_atoms

N0 = 10000
total_years = 50000

decay = decay(N0, decay_probability, total_years)
t = list(range(total_years + 1))
```

```
theoretical_decay = N0 * np.exp(-decay_constant * np.array(time_steps))

plt.figure(figsize=(10, 6))
plt.plot(t, decay, label='Monte Carlo Simulation', color='green')
plt.plot(t, theoretical_decay, label='Theoretical Curve', color='red', linestyle='-
plt.title(f'Radioactive Decay of Carbon-14 (N₀ = {N0})')
plt.xlabel('Time (years)')
plt.ylabel('Number of Undecayed Atoms')
plt.legend()
plt.grid()
plt.show()
```



The simulated curve is very similar to theoretical curve. So the Monte Carlo simulation gives the same curve when compare with the theoretical curve.

In [ ]: